NAME : PENDEM HARSHITHA

ROLLNO: 2403A510C9

BATCHNO: 05

COURSE: AI ASSISSTED CODING .

Task-1: Ask AI to generate a Flask REST API with one route:
GET /hello → returns {"message": "Hello, AI Coding!"}

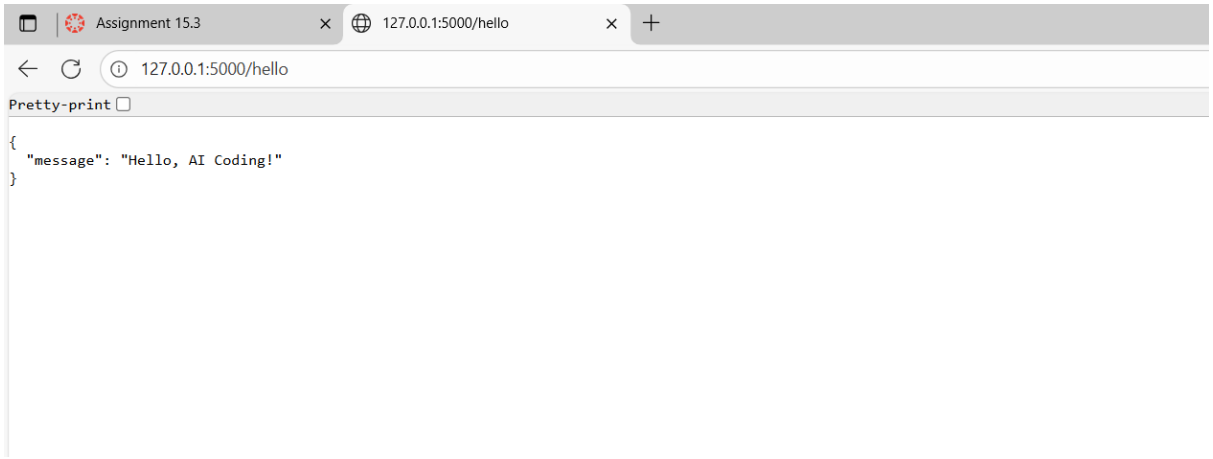
PROMPT:

"Generate a simple Flask REST API in Python. The API should have a single endpoint: `GET /hello`. When this endpoint is accessed, it must return a JSON response with the following structure: `{\"message\": \"Hello, AI Coding!\"}`. Please include comments in the code and instructions on how to run the application."

CODE:

```python
# app.py
from flask import Flask, jsonify

# Initialize the Flask application instance
app = Flask(__name__)

@app.route('/hello', methods=['GET'])
def get_hello_message():
    """
    Defines the endpoint for GET /hello.
    Returns a static JSON message.
    """
    # Create a Python dictionary for the response
    message = {
        "message": "Hello, AI Coding!"
    }
    # Use Flask's jsonify to convert the dictionary to a JSON response
    return jsonify(message)

# This block allows the script to be run directly
# The server will only run if this file is executed as the main program
if __name__ == '__main__':
    # Run the app in debug mode for development
    # It will listen on http://127.0.0.1:5000 by default
    app.run(debug=True)
```

OUTPUT:



OBSERVATION:

- **Simplicity and Readability:** It uses standard Flask decorators (`@app.route`) to clearly map a URL endpoint to a Python function.
- **Correct JSON Handling:** It correctly uses the `jsonify` utility from Flask. This is important because `jsonify` not only converts a Python dictionary to a JSON string but also sets the correct `Content-Type` header to `application/json`, which is the standard for REST APIs.
- **Standard Structure:** The `if __name__ == '__main__':` block is a standard Python convention that makes the script reusable and prevents the server from running if the file is imported into another module.
- **Extensibility:** This simple file serves as an excellent foundation. You can easily add more routes by defining new functions with their own `@app.route` decorators, connect to a database, or add more complex business logic.

Task-2:

Use AI to build REST endpoints for a Student API:

• GET /students → List all students.

• POST /students → Add a new student.

• PUT /students/<id> → Update student details.

• DELETE /students/<id> → Delete a student.

Expected Output:

• Flask API with dictionary/list storage.

• JSON responses for each operation

Prompt:

"Generate a complete Flask REST API to manage a collection of students. The student data should be stored in-memory using a Python list of dictionaries.

Please implement the following four endpoints:

1. **GET /students**: Returns a JSON array of all students.
2. **POST /students**: Accepts a JSON object like {\"name\": \"New Student\", \"major\": \"Computer Science\"}. It should generate a new unique ID, add the student to the list, and return the complete student object (including the new ID) with a 201 Created status code.
3. **PUT /students/<int:student_id>**: Finds a student by their ID. It should update the student's data with the provided JSON payload and return the updated student object. If the student ID is not found, it should return a 404 error with a message.
4. **DELETE /students/<int:student_id>**: Finds a student by their ID and removes them from the list. It should return a success message. If the student ID is not found, it should return a 404 error.

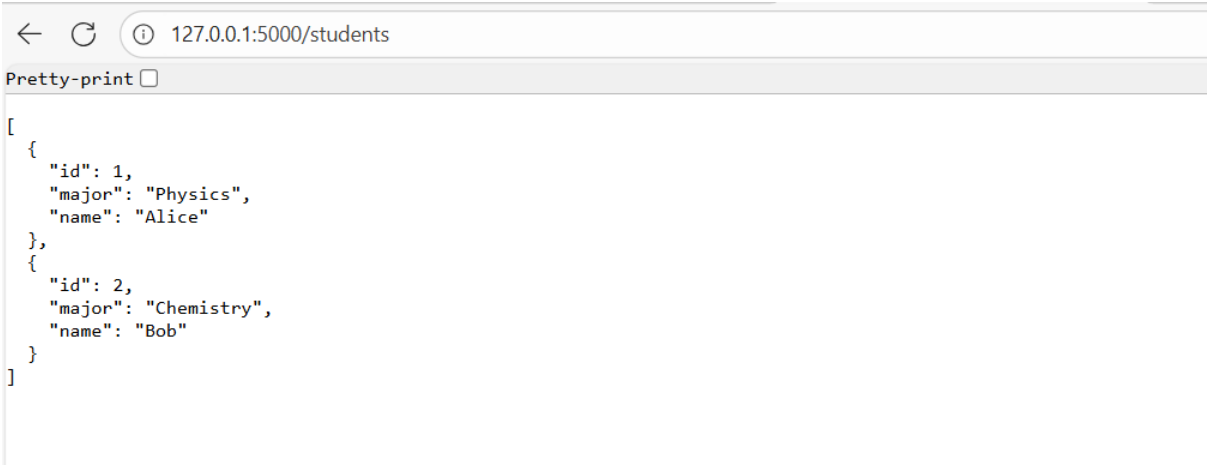
Code:

```python
# app.py
from flask import Flask, jsonify, request

# Initialize the Flask application instance
app = Flask(__name__)

# In-memory data store (simulating a database)
students = [
    {"id": 1, "name": "Alice", "major": "Physics"},
    {"id": 2, "name": "Bob", "major": "Chemistry"}
]
# A simple counter to generate unique IDs for new students
next_id = 3

@app.route('/students', methods=['GET'])
def get_students():
    """
    GET /students
    Returns the list of all students.
    """
    return jsonify(students)

@app.route('/students', methods=['POST'])
def add_student():
    """
    POST /students
    Adds a new student to the list.
    """
    global next_id
    if not request.json or 'name' not in request.json or 'major' not in request.json:
        return jsonify({"error": "Missing name or major in request body"}), 400
```

```
    new_student = {
        "id": next_id,
        "name": request.json['name'],
        "major": request.json['major']
    }
    students.append(new_student)
    next_id += 1
    return jsonify(new_student), 201 # 201 Created

@app.route('/students/<int:student_id>', methods=['PUT'])
def update_student(student_id):
    """
    PUT /students/<id>
    Updates an existing student's details.
    """
    student = next((s for s in students if s['id'] == student_id), None)
    if student is None:
        return jsonify({"error": "Student not found"}), 404

    # Update fields if they exist in the request JSON
    student['name'] = request.json.get('name', student['name'])
    student['major'] = request.json.get('major', student['major'])

    return jsonify(student)

@app.route('/students/<int:student_id>', methods=['DELETE'])
def delete_student(student_id):
    """
    DELETE /students/<id>
    Deletes a student from the list.
    """
```

```
    DELETE /students/<id>
    Deletes a student from the list.
    """
    global students
    student = next((s for s in students if s['id'] == student_id), None)
    if student is None:
        return jsonify({"error": "Student not found"}), 404

    # Re-create the list excluding the student to be deleted
    students = [s for s in students if s['id'] != student_id]
    return jsonify({"message": f"Student with id {student_id} deleted successfully"})

# This block allows the script to be run directly
# The server will only run if this file is executed as the main program
if __name__ == '__main__':
    # Run the app in debug mode for development
    # It will listen on http://127.0.0.1:5000 by default
    app.run(debug=True, port=5000)
```

Output:



Observation:

- **RESTful Principles:** The code correctly uses HTTP methods (`GET`, `POST`, `PUT`, `DELETE`) to represent actions on the `students` resource. The URL structure is clean and follows standard conventions.
- **State Management:** It uses a simple global list (`students`) and a counter (`next_id`) for in-memory storage, which is perfect for a self-contained demo.
- **Error Handling:** The code includes essential error handling, such as returning a `404 Not Found` status when an ID doesn't exist and a `400 Bad Request` if the `POST` payload is malformed. This makes the API more reliable.
- **Correct HTTP Status Codes:** It correctly returns `201 Created` for a successful `POST` operation, which is a key detail in REST API design, signaling that a new resource was created.
- **Readability and Maintainability:** The code is well-commented, with each function having a docstring that explains its purpose and the endpoint it serves. This makes it easy to understand and extend.

Task-3: Ask AI to generate a REST API endpoint

Expected Output:

Working search function with query param handling.

Prompt:

"Please extend the existing Flask REST API for students by adding a new search endpoint.

The new endpoint should be: **GET /students/search**

It must accept a URL query parameter named `q`. The endpoint should perform a case-insensitive search and return a JSON list of all students where the search term is found in either the student's `name` or `major`.

If the `q` parameter is missing or empty, or if no students match the search, it should return an empty JSON array `[]`.

Please provide the updated code and a `curl` example to test the search functionality."

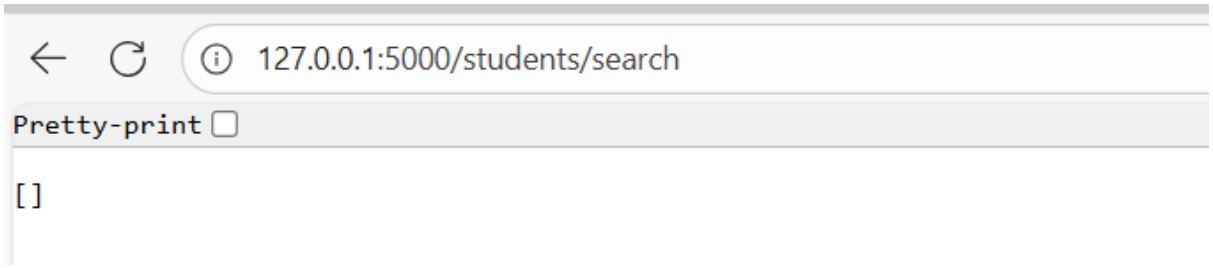
Code:

```python
# app.py
from flask import Flask, jsonify, request

# Initialize the Flask application instance
app = Flask(__name__)

# In-memory data store (simulating a database)
students = [
    {"id": 1, "name": "Alice", "major": "Physics"},
    {"id": 2, "name": "Bob", "major": "Chemistry"}
]
# A simple counter to generate unique IDs for new students
next_id = 3

@app.route('/students', methods=['GET'])
def get_students():
    """
    GET /students
    Returns the list of all students.
    """
    return jsonify(students)

@app.route('/students/search', methods=['GET'])
def search_students():
    """
    GET /students/search?q=<query>
    Searches for students by name or major (case-insensitive).
    """
    # Get the search query from URL parameters, default to empty string
    query = request.args.get('q', '').lower()
    if not query:
        return jsonify([]) # Return empty list if no query is provided
```

```python
    # Filter students based on the query
    search_results = [s for s in students if
                      query in s['name'].lower() or query in s['major'].lower()]
    return jsonify(search_results)

@app.route('/students', methods=['POST'])
def add_student():
    """
    POST /students
    Adds a new student to the list.
    """
    global next_id
    if not request.json or 'name' not in request.json or 'major' not in request.json:
        return jsonify({"error": "Missing name or major in request body"}), 400

    new_student = {
        "id": next_id,
        "name": request.json['name'],
        "major": request.json['major']
    }
    students.append(new_student)
    next_id += 1
    return jsonify(new_student), 201 # 201 Created

@app.route('/students/<int:student_id>', methods=['PUT'])
def update_student(student_id):
    """
    PUT /students/<id>
    Updates an existing student's details.
    """
    student = next((s for s in students if s['id'] == student_id), None)
    if student is None:
```

```python
    if student is None:
        return jsonify({"error": "Student not found"}), 404

    # Update fields if they exist in the request JSON
    student['name'] = request.json.get('name', student['name'])
    student['major'] = request.json.get('major', student['major'])

    return jsonify(student)

@app.route('/students/<int:student_id>', methods=['DELETE'])
def delete_student(student_id):
    """
    DELETE /students/<id>
    Deletes a student from the list.
    """
    global students
    student = next((s for s in students if s['id'] == student_id), None)
    if student is None:
        return jsonify({"error": "Student not found"}), 404

    # Re-create the list excluding the student to be deleted
    students = [s for s in students if s['id'] != student_id]
    return jsonify({"message": f"Student with id {student_id} deleted successfully"})

# This block allows the script to be run directly
# The server will only run if this file is executed as the main program
if __name__ == '__main__':
    # Run the app in debug mode for development
    # It will listen on http://127.0.0.1:5000 by default
    app.run(debug=True, port=5000)

```

Output:



Observation:

- **Query Parameter Handling:** The code correctly uses `request.args.get('q', '')` to safely retrieve the query parameter. This method is robust because it doesn't raise an error if the `q` parameter is missing; instead, it defaults to an empty string.
- **Case-Insensitive Search:** The implementation correctly converts both the search query and the student data (`name`, `major`) to lowercase before comparison, ensuring the search is case-insensitive as requested.
- **Efficient Filtering:** A list comprehension (`[s for s in students if ...]`) is used to filter the data. This is a concise and highly "Pythonic" way to create a new list based on a condition, making the code readable and efficient for in-memory data.
- **RESTful Design:** The new endpoint `GET /students/search` is a good design choice. It clearly separates the action of searching from fetching the entire collection (`GET /students`), which is a common and clean pattern in REST API design.

Task-4: Ask AI to write test scripts using Python requests module to call APIs created above.

Prompt:

"Write a Python script to test the Flask Student API. The script must use the `requests` library to make HTTP calls to a locally running server at `http://127.0.0.1:5000`.

Code:

```python
# test_api.py
import requests
import json

# The base URL for the API, assuming it's running locally on port 5000
BASE_URL = "http://127.0.0.1:5000"

def run_tests():
    """
    Executes a sequence of API tests for the Student API.
    """
    print("--- Starting API Integration Tests ---")

    # This will store the ID of the student we create
    new_student_id = None

    try:
        # 1. GET /students: Fetch initial list
        print("\n[TEST 1] GET /students - Fetching all students...")
        response = requests.get(f"{BASE_URL}/students")
        assert response.status_code == 200
        initial_students = response.json()
        assert len(initial_students) == 2
        print("PASS: Initial student list fetched successfully.")

        # 2. POST /students: Add a new student
        print("\n[TEST 2] POST /students - Adding a new student...")
        new_student_data = {"name": "David", "major": "Mathematics"}
        response = requests.post(f"{BASE_URL}/students", json=new_student_data)
        assert response.status_code == 201
        created_student = response.json()
        new_student_id = created_student.get("id")
        assert new_student_id is not None
        assert created_student["name"] == "David"
        print(f"PASS: Student 'David' created with ID: {new_student_id}")

        # 3. GET /students/search: Search for the new student
        print("\n[TEST 3] GET /students/search - Searching for 'david'...")
        response = requests.get(f"{BASE_URL}/students/search?q=david")
        assert response.status_code == 200
        search_results = response.json()
        assert len(search_results) == 1
        assert search_results[0]["name"] == "David"
        print("PASS: Search found the new student correctly.")

        # 4. PUT /students/<id>: Update the student's major
        print(f"\n[TEST 4] PUT /students/{new_student_id} - Updating student's major...")
        update_data = {"major": "Applied Mathematics"}
        response = requests.put(f"{BASE_URL}/students/{new_student_id}", json=update_data)
        assert response.status_code == 200
        updated_student = response.json()
        assert updated_student["major"] == "Applied Mathematics"
        print("PASS: Student's major updated successfully.")

        # 5. DELETE /students/<id>: Delete the student
        print(f"\n[TEST 5] DELETE /students/{new_student_id} - Deleting the student...")
        response = requests.delete(f"{BASE_URL}/students/{new_student_id}")
        assert response.status_code == 200
        delete_message = response.json()
        assert "deleted successfully" in delete_message.get("message", "")
        print("PASS: Student deleted successfully.")

        # 6. Error Case: Verify deletion by trying to get the student again
        print(f"\n[TEST 6] GET /students/{new_student_id} - Verifying deletion (expect 404)...")
        # We need to find the student in the list to confirm it's gone
```

```
        assert "deleted successfully" in delete_message.get("message", "")
        print("PASS: Student deleted successfully.")

        # 6. Error Case: Verify deletion by trying to get the student again
        print(f"\n[TEST 6] GET /students/{new_student_id} - Verifying deletion (expect 404)...")
        # We need to find the student in the list to confirm it's gone
        response = requests.get(f"{BASE_URL}/students")
        all_students = response.json()
        student_exists = any(s['id'] == new_student_id for s in all_students)
        assert not student_exists
        print("PASS: Student is confirmed to be removed from the list.")

        # Bonus Test: Try to update a non-existent student
        print(f"\n[BONUS TEST] PUT /students/999 - Updating non-existent student (expect 404)...")
        response = requests.put(f"{BASE_URL}/students/999", json={"name": "Ghost"})
        assert response.status_code == 404
        print("PASS: Correctly received 404 for non-existent student update.")

    except (requests.exceptions.ConnectionError, AssertionError) as e:
        print(f"\n--- A TEST FAILED ---")
        print(f"Error: {e}")
        print("Please ensure the Flask API server is running.")
        return

    print("\n--- All tests passed successfully! ---")

if __name__ == "__main__":
    run_tests()
```

Output:

```
PS C:\Users\pende\OneDrive\Desktop\wt2> & C:/Users/pende/anaconda3/python.exe c:/Users/pende/OneDrive/Deskto
p/wt2/17.1.py
--- Starting API Integration Tests ---

[TEST 1] GET /students - Fetching all students...

--- A TEST FAILED ---
Error: HTTPConnectionPool(host='127.0.0.1', port=5000): Max retries exceeded with url: /students (Caused by
NewConnectionError('<urllib3.connection.HTTPConnection object at 0x00000152F0410D70>: Failed to establish a
new connection: [WinError 10061] No connection could be made because the target machine actively refused it'
))
Please ensure the Flask API server is running.
```

Observation:

- **Clarity and Structure:** The script is organized into a clear, sequential flow. Each test step is announced with a `print` statement, making the output easy to follow.
- **Use of `requests`:** It correctly uses the `requests` library's methods (`get`, `post`, `put`, `delete`) and passes JSON data where required.
- **Assertions:** The use of `assert` is a standard and effective way to verify test outcomes. It checks for correct HTTP status codes and validates the content of the JSON responses, immediately halting the script if a condition is not met.
- **State Management:** The script correctly handles the stateful nature of the API by capturing the `id` of the newly created student and using it in subsequent `PUT` and `DELETE` requests.
- **Covers Success and Failure:** It tests the "happy path" (creating, updating, deleting successfully) as well as important error cases (verifying deletion, attempting to update a non-existent record), making the test suite more robust.
- **Practicality:** This script is a practical, real-world example of how developers write automated tests to ensure their APIs function as expected, preventing regressions when new features are added or code is refactored.